



Informatique graphique avancée, animation et rendu

Compte-rendu TP4

Dual contouring

Louis Jean

Master 2 IMAGINE
Université de Montpellier
N° étudiant : 21914083

2 octobre 2024

Table des matières

1	Introduction	2
2	Construction de la grille	3
3	Fonction de distance signée (SDF)	4
4	Dual contouring	5
4.1	Visualisation des points	5
4.1.1	Point au centre de la cellule	5
4.1.2	Point reprojeté sur la surface MLS	8
4.2	Extraction d'un maillage triangulaire	9
5	Conclusion	10

1 Introduction

Dans ce TP, nous nous sommes intéressés à l'implémentation de l'algorithme de dual contouring pour la reconstruction de surfaces à partir de nuages de points. Ce TP s'appuie sur la base de code développée lors du premier TP, où nous avons implémenté la projection de points sur une surface MLS notamment à l'aide de la méthode HPSS.

L'objectif principal de ce TP est de construire une grille régulière englobant le nuage de points, de calculer une fonction de distance signée à chaque point de la grille, puis d'appliquer l'algorithme de dual contouring pour extraire une représentation surfacique du nuage de points.

Quelques instructions pour le programme :

- Pour afficher / masquer la grille : touche G
- Pour afficher / masquer les points issus du dual contouring : touche D
- Pour afficher / masquer les cellules candidates : touche M
- Pour afficher / masquer le nuage de points original : touche 1

2 Construction de la grille

Pour conceptualiser une grille régulière, j'ai créé deux structures. `Cell` représente une cellule de la grille, et contient un booléen indiquant si la cellule possède un changement de signe, et un tableau de 8 paires de point-distance, chaque point étant un coin de la cellule, auquel correspond la distance à la surface calculée par la Signed Distance Function (SDF) que nous verrons plus tard. `Grid` représente la grille entière. Elle est composée d'une liste de cellules, des résolutions sur chaque axe, et des tailles d'une cellule sur chaque axe.

```
1 struct Cell {
2     std::array<std::pair<Vec3,float>, 8> corners;
3     bool isCandidate = false;
4 };
5
6 struct Grid {
7     std::vector<Cell> cells;
8     int dimX, dimY, dimZ;
9     float dx, dy, dz; // Ici car grille régulière
10    void buildGrid(const std::vector<Vec3>& positions, std::vector<Vec3>& normals,
11                  ↪ BasicANNkdTree& kdtree, KernelType kernel_type, float radius);
12    void drawGrid();
13 };
14
```

Pour construire la grille, j'ai d'abord parcouru l'espace de manière à créer 8 sommets par cellule.

```
1 void Grid::buildGrid(const std::vector<Vec3>& positions, std::vector<Vec3>& normals,
2 ↪ BasicANNkdTree& kdtree, KernelType kernel_type, float radius) {
3     Vec3 BBmin = minVectorVec3(positions);
4     Vec3 BBmax = maxVectorVec3(positions);
5     BBmin -= 0.01f;
6     BBmax += 0.01f;
7     grid.dx = (BBmax[0] - BBmin[0]) / grid.dimX;
8     grid.dy = (BBmax[1] - BBmin[1]) / grid.dimY;
9     grid.dz = (BBmax[2] - BBmin[2]) / grid.dimZ;
10    grid.cells.resize(grid.dimX*grid.dimY*grid.dimZ);
11    for(int i = 0; i < grid.dimX; ++i) {
12        for(int j = 0; j < grid.dimY; ++j) {
13            for(int k = 0; k < grid.dimZ; ++k) {
14                Cell c;
15                for(int di = 0; di < 2; ++di) {
16                    for(int dj = 0; dj < 2; ++dj) {
17                        for(int dk = 0; dk < 2; ++dk) {
18                            Vec3 currentCorner(BBmin[0] + (i+di)*grid.dx,
19                                                BBmin[1] + (j+dj)*grid.dy,
20                                                BBmin[2] + (k+dk)*grid.dz);
21                            ...
22                        }
23                    }
24                }
25            }
26        }
27    }
28 }
```

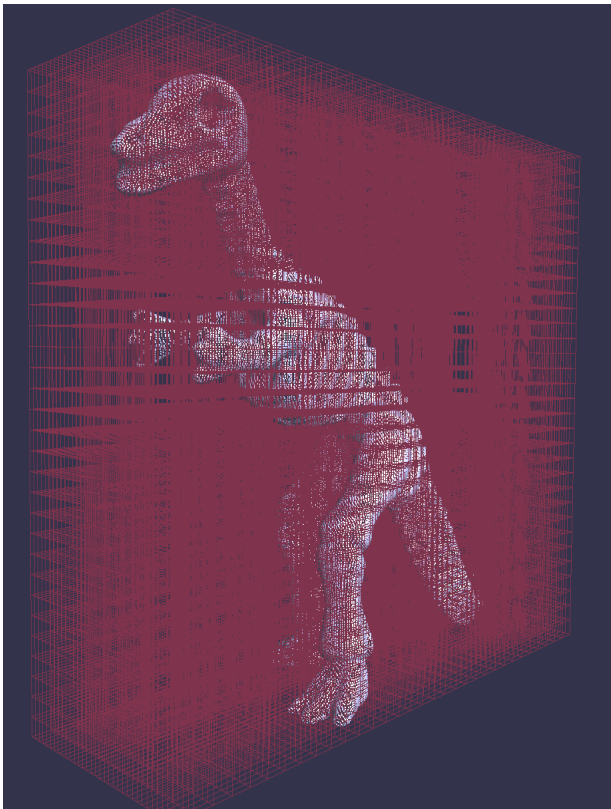
Ensuite, j'ai calculé la distance signée à la surface MLS pour chacun des points de chaque cellule. Nous discuterons de l'implémentation de la SDF juste après.

```
1 float distance = computeSDF(currentCorner, positions, normals, kdtree, kernel_type, radius);
```

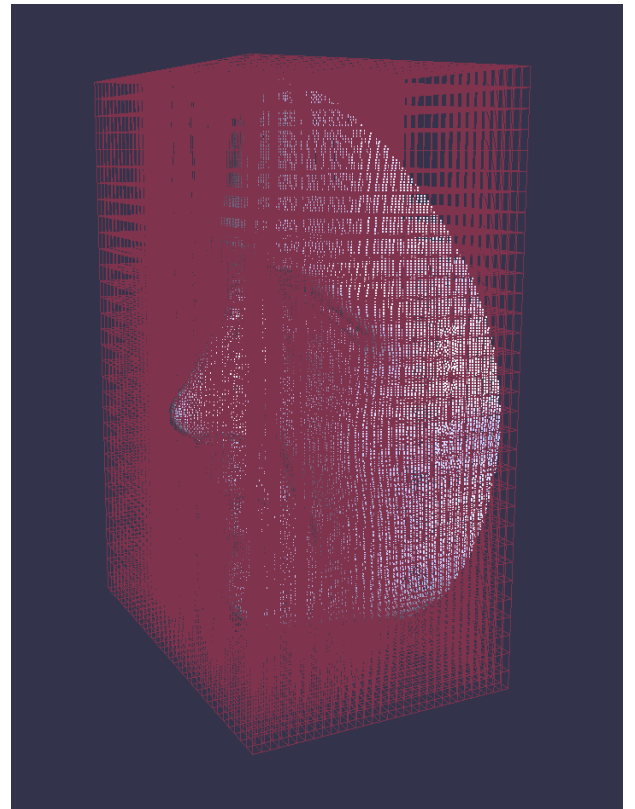
Enfin, j'ai alloué mes cellules avec les données calculées.

```
1 c.corners[(di*4)+(dj*2)+dk] = std::make_pair(currentCorner, distance);
2 grid.cells[i * grid.dimY * grid.dimZ + j * grid.dimZ + k] = c;
```

J'ai choisi de faire mes calculs de distance directement à la création de la grille, ce n'est pas optimal en terme de lisibilité de code mais c'était le plus rapide à implémenter. Voici les résultats que l'on peut observer lorsque l'on affiche la grille.



(a) Grille encadrant le nuage de points *dino.pn*



(b) Grille encadrant le nuage de points *face.pn*

Figure 1: Aperçu de grilles 32x32x32

3 Fonction de distance signée (SDF)

Une fonction de distance signée permet de définir une surface à l'aide du 0-set de la fonction. Dans notre cas, elle s'exprime ainsi :

$$f(x) = (x - p(x))^T n(x)$$

avec $p(x)$ la projection HPSS du point x , $n(x)$ la normale au point x .

Si $f(x) > 0$, alors le point est à l'extérieur de la surface. Si $f(x) < 0$, alors le point est à l'intérieur de la surface. Enfin, si $f(x) = 0$, alors le point est sur la surface. L'ensemble $\{x | f(x) = 0\}$, qui est nommé le 0-set de la fonction, définit la surface.

Voici comment j'ai implémenté cette fonction dans mon programme.

```
1 float computeSDF(Vec3& gridPoint, const std::vector<Vec3>& positions, const std::vector<Vec3>&
  ↪ normals, BasicANNkdTree& kdtree, KernelType kernel_type, float radius) {
2     Vec3 projectedPoint, projectedNormal;
3     HPSS(gridPoint, projectedPoint, projectedNormal, positions, normals, kdtree, kernel_type,
  ↪ radius);
4     float distance = (gridPoint - projectedPoint).length();
5     if (Vec3::dot(gridPoint - projectedPoint, projectedNormal) < 0.0f) {
6         distance = -distance;
7     }
8     return distance;
9 }
```

Ici, la fonction HPSS est la même que celle implémentée au premier TP de cette UE.

4 Dual contouring

À partir des points de la grille et des distances signées, on peut appliquer le dual contouring. L'idée est de créer un nouveau sommet dans chaque cellule pour laquelle on observe un changement de signe parmi ses coins.

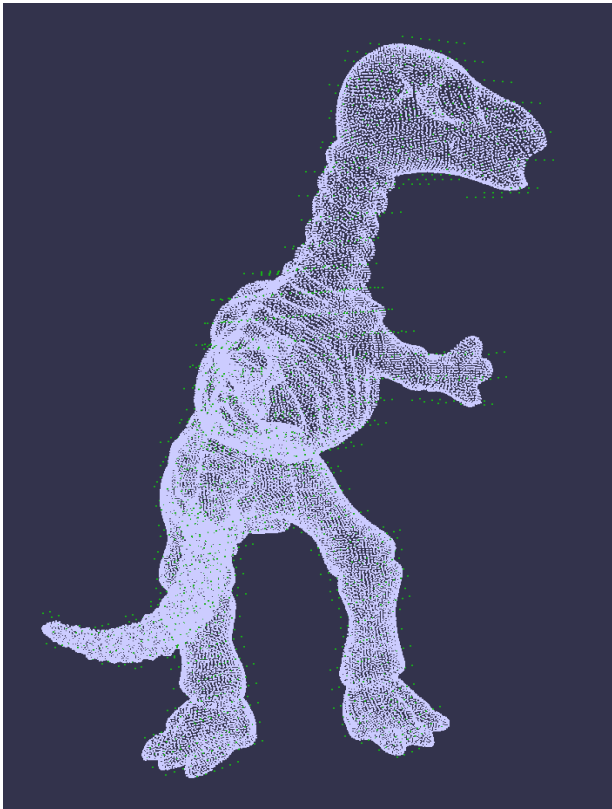
4.1 Visualisation des points

4.1.1 Point au centre de la cellule

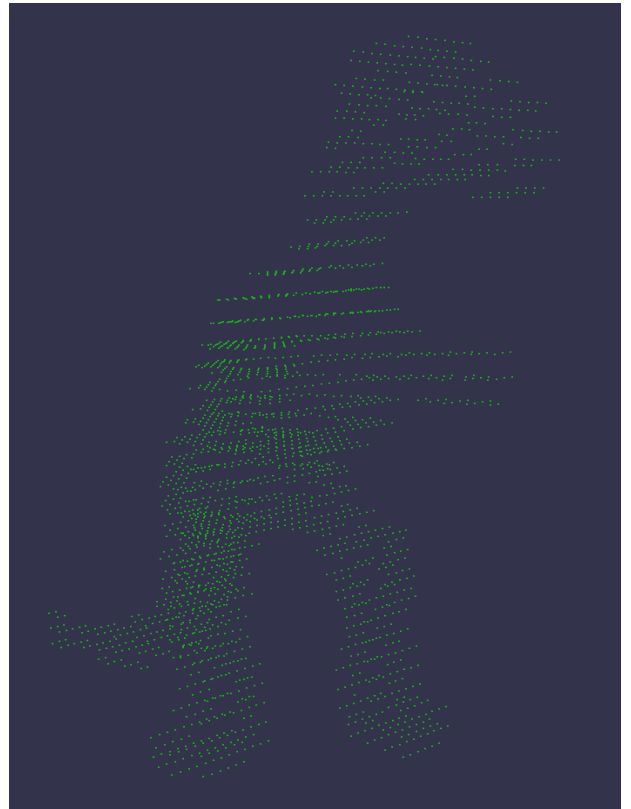
Dans un premier temps, on crée un nouveau sommet et on le place en plein centre de la cellule, en faisant une moyenne des coins de la cellule.

```
1 void dualContouringBarycentre(Grid& grid, std::vector<Vec3>& dualContouringPoints) {
2     for(auto& cell : grid.cells) {
3         bool hasPositive = false;
4         bool hasNegative = false;
5         for(int i = 0; i < 8; ++i) {
6             float sdf = cell.corners[i].second;
7             if(sdf < 0.0) {
8                 hasNegative = true;
9             }
10            if(sdf > 0.0) {
11                hasPositive = true;
12            }
13        }
14        if(hasNegative && hasPositive) {
15            Vec3 bary(0.0f,0.0f,0.0f);
16            for(int i = 0; i < 8; ++i) {
17                bary += cell.corners[i].first;
18            }
19            bary /= 8;
20            dualContouringPoints.push_back(bary);
21            cell.isCandidate = true;
22        }
23    }
```

Avec cette méthode, on obtient déjà des résultats assez satisfaisants.

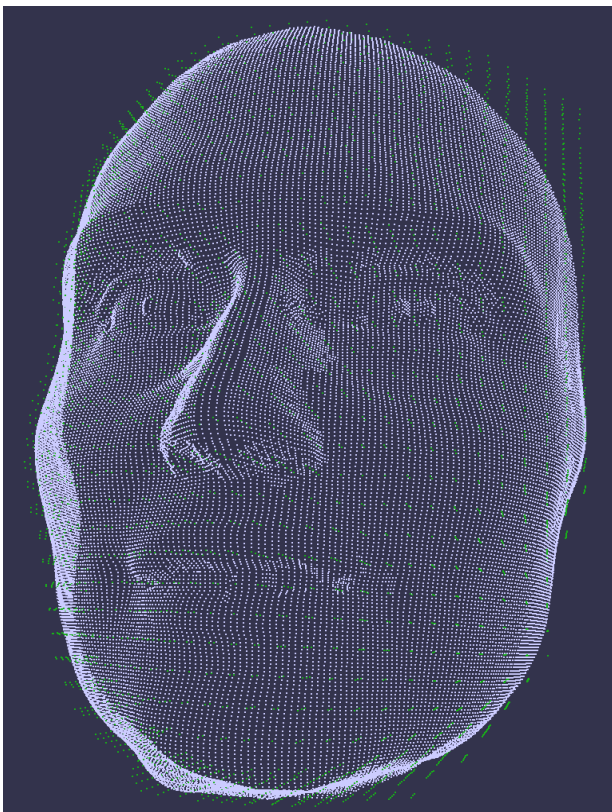


(a) Nuage de points *dino.pn* original

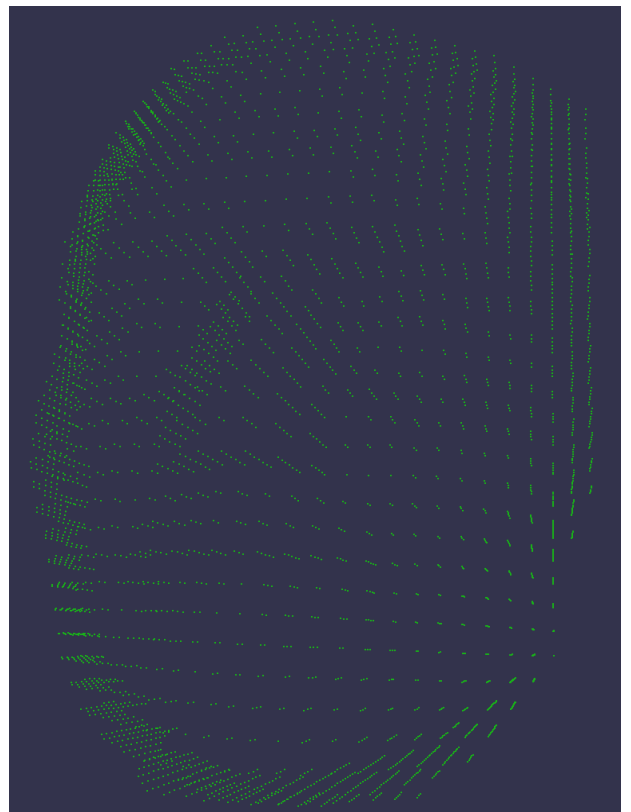


(b) Points issus du dual contouring sur *dino.pn*

Figure 2: Dual contouring avec grille 32x32x32 sur *dino.pn*



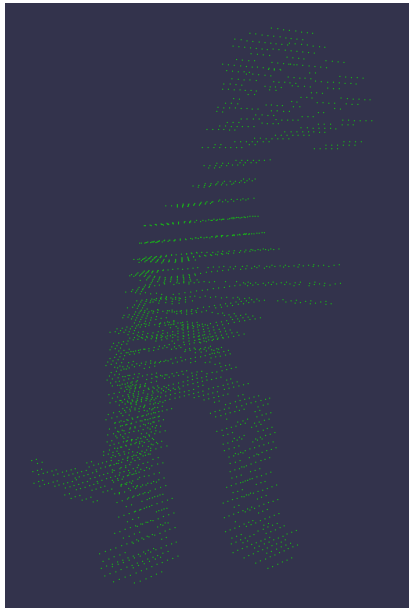
(a) Nuage de points *face.pn* original



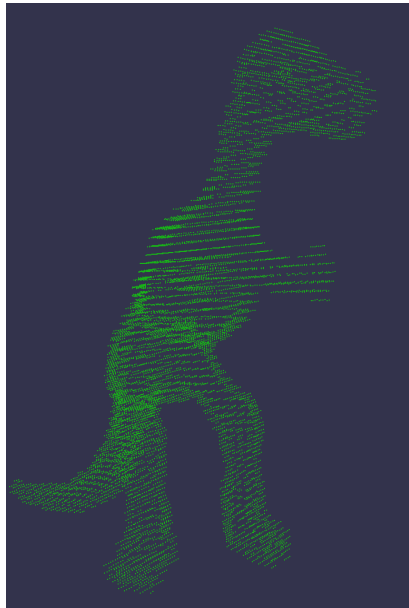
(b) Points issus du dual contouring sur *face.pn*

Figure 3: Dual contouring avec grille 32x32x32 sur *face.pn*

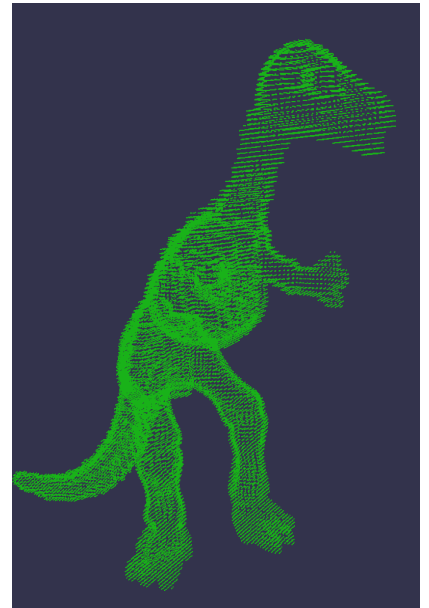
J'ai aussi fait quelques tests pour des grilles de différentes tailles.



(a) Grille 32x32x32

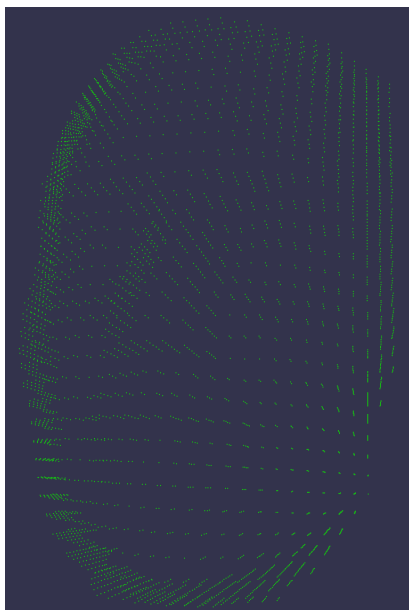


(b) Grille 64x64x64

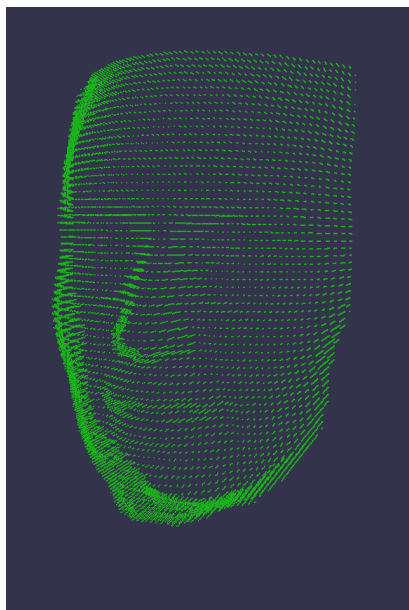


(c) Grille 128x128x128

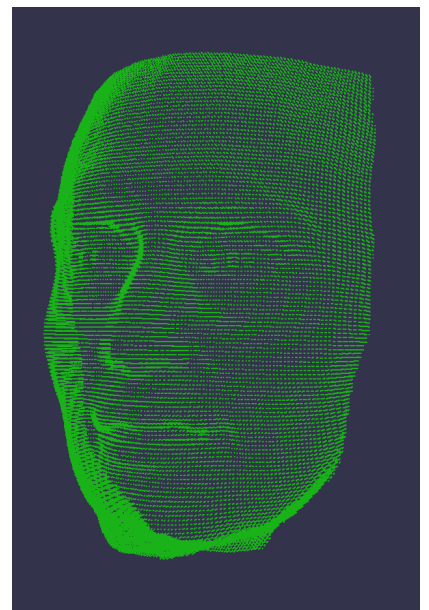
Figure 4: Comparaison des résultats avec différentes tailles de grilles pour *dino.pn*



(a) Grille 32x32x32



(b) Grille 64x64x64



(c) Grille 128x128x128

Figure 5: Comparaison des résultats avec différentes tailles de grilles pour *face.pn*

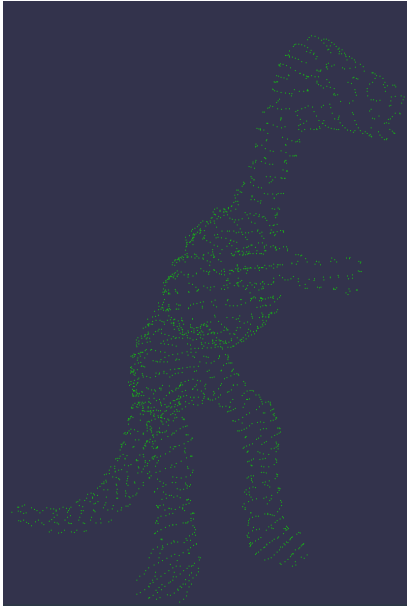
En termes de performances, sur ma machine :

- Pour une résolution de 32, le programme met environ 5 secondes à calculer le dual contouring.
- Pour une résolution de 64, on passe à environ 40 secondes.
- Pour une résolution de 128, on passe à environ 3 minutes.

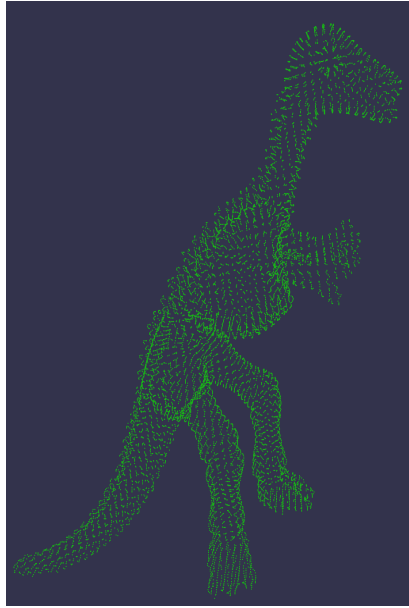
Je pense qu'une manière d'améliorer les performances serait d'éviter la duplication de sommets dans les cellules, car actuellement chaque cellule comporte 8 sommets, même si beaucoup d'entre eux sont partagés par les cellules voisines à chaque fois, ce qui cause de la redondance dans les calculs.

4.1.2 Point reprojeté sur la surface MLS

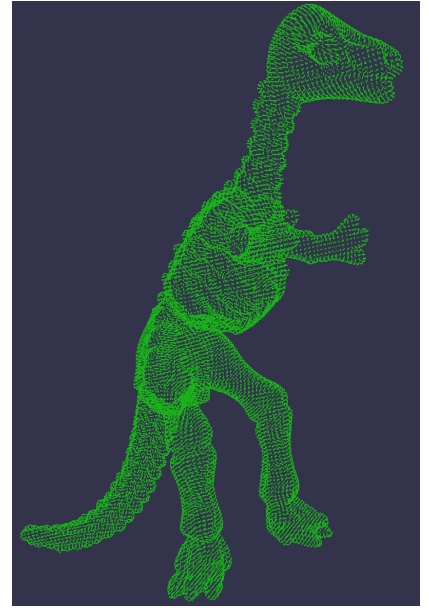
Mais l'on peut faire mieux comme visualisation ! Si lorsque l'on a les points issus du dual contouring (les barycentres des cellules élues), on reprojette ces derniers sur la surface MLS, alors on obtient un meilleur résultat.



(a) Grille 32x32x32

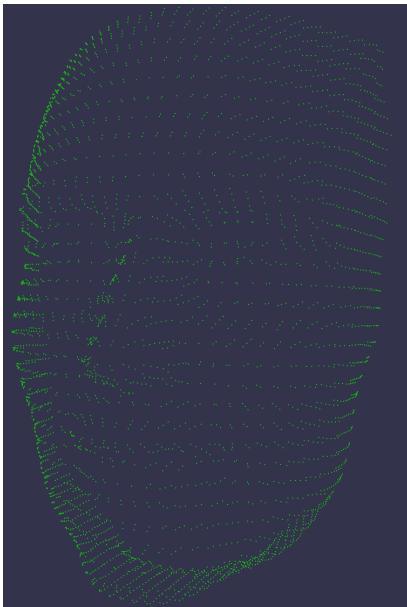


(b) Grille 64x64x64

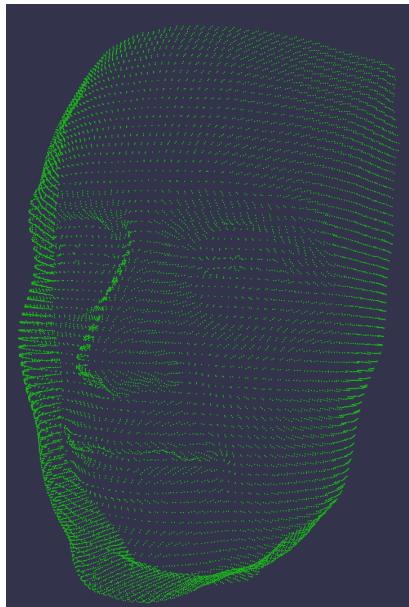


(c) Grille 128x128x128

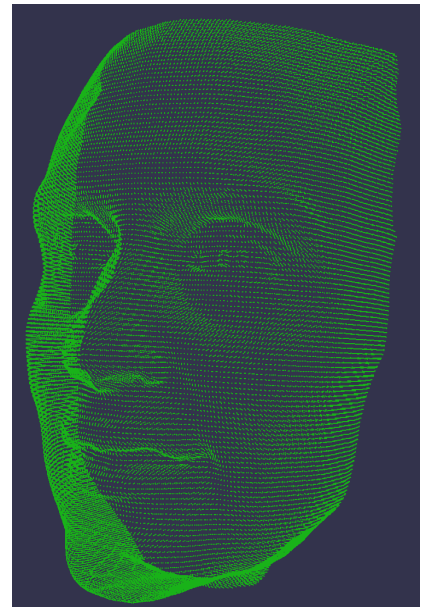
Figure 6: Comparaison des résultats avec différentes tailles de grilles **et reprojection des barycentres sur la surface MLS** pour *dino.pn*



(a) Grille 32x32x32



(b) Grille 64x64x64



(c) Grille 128x128x128

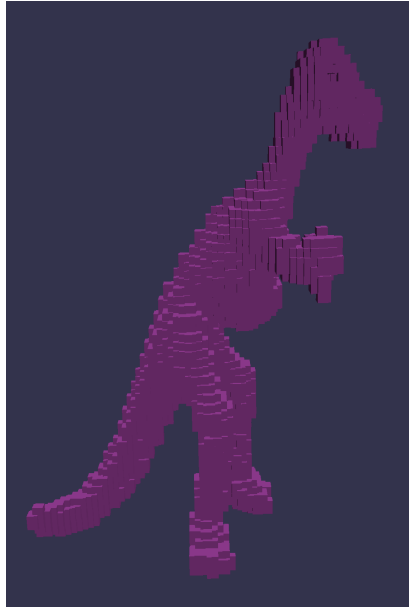
Figure 7: Comparaison des résultats avec différentes tailles de grilles **et reprojection des barycentres sur la surface MLS** pour *face.pn*

4.2 Extraction d'un maillage triangulaire

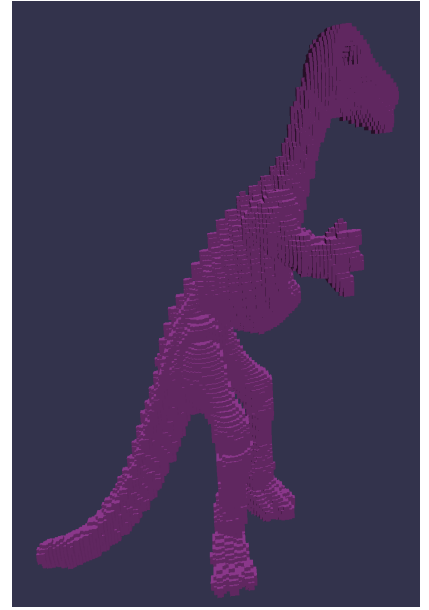
À partir des points déduits par le dual contouring, on veut pouvoir créer un maillage. Pour cela, il faut pour chaque arête possédant un changement de signe, relier les sommets des cellules voisines par cette arête. Malheureusement, je n'ai pas prévu de structure de données assez modulaire pour permettre cela avec mon implémentation actuelle, et je manque de temps pour corriger ce problème, car plus j'ai fait de recherches, plus cela s'est avéré compliqué à mettre en place. Cependant, à défaut d'avoir réussi cette étape, j'ai décidé de dessiner les 6 faces de chaque cellule candidate (celles qui contiennent un changement de signe dans leurs coins) pour obtenir un semblant de maillage, et je trouve le résultat assez convaincant (même si pas exploitable dans de vraies applications).



(a) Grille 32x32x32

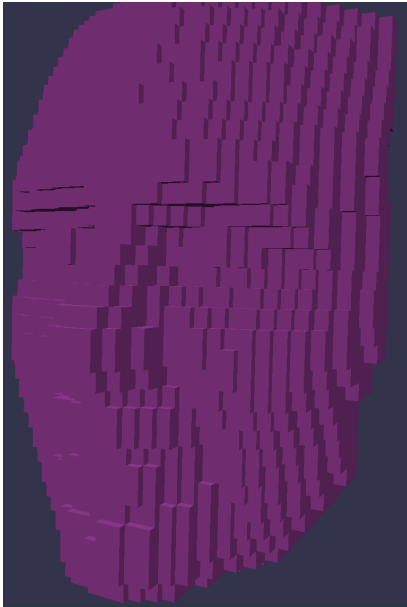


(b) Grille 64x64x64

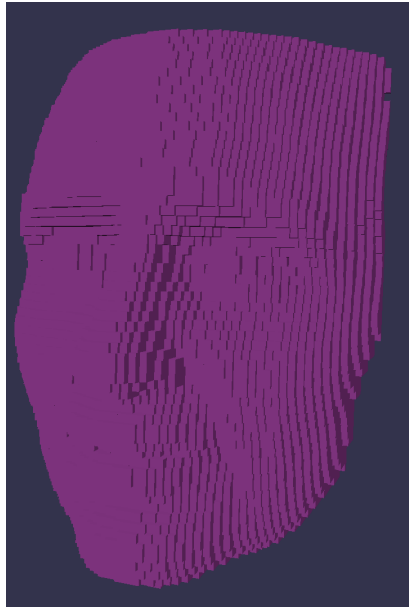


(c) Grille 128x128x128

Figure 8: Cellules candidates dessinées pour *dino.pn*



(a) Grille 32x32x32



(b) Grille 64x64x64



(c) Grille 128x128x128

Figure 9: Cellules candidates dessinées pour *face.pn*

5 Conclusion

Ce TP a mis en évidence la puissance et la complexité de l'algorithme de dual contouring. Les résultats obtenus montrent que, même avec des grilles de résolution modérée, il est possible de reconstruire efficacement la forme générale des objets à partir de nuages de points.

Personnellement, j'ai beaucoup aimé ce TP malgré les difficultés rencontrées, je trouve cette technique très ingénieuse bien que compliquée (on ne fait pas de feu sans fumée), et je pense en implémenter une version de mon côté avec plus de temps, car le potentiel est vraiment très grand.

Merci pour le temps et l'attention que vous avez consacrés à la lecture de ce compte-rendu.